

Implementing and Testing a Probing-Based Path Verification for PolKA Source Routing Protocol

Henrique C. Layber, Roberta L. Gomes, Magnos Martinello

¹Department of Informatics

Federal University of Espírito Santo (UFES) – Vitória, ES – Brazil

henrique.layber@edu.ufes.br, {roberta.gomes, magnos.martinello}@ufes.br

Abstract. *Presents an implementation of a route probing and validation mechanism for the source routing protocol PolKA. The mechanism is based on a composition of checksum functions on stateless core switches, which allows a trusted party to verify if a packet traversed the network along the source-defined path.*

Resumo. *Este trabalho apresenta uma implementação de um mecanismo de sondagem e verificação de rota para um protocolo de roteamento na origem chamado PolKA. O mecanismo é baseado em uma combinação de funções de hash, calculadas por switches de núcleo sem estado, permitindo a uma parte confiável verificar se um pacote percorreu a rede ao longo do caminho definido pela origem.*

Comment

Checar se keywords estão boas para o SBC. Só copieei do **PathSec**

Keywords – Path Aware; Network Security; Path Verification; In-networking Programming

1. Introduction

Ever since *Source Routing* (SR) was proposed, there has been a need to ensure that packets traverse the network along the paths selected by the source, not only for security reasons but also to ensure that the network is functioning correctly and correctly configured. This is particularly important in the context of *Software-Defined Networks* (SDNs), where the control plane can select paths based on a variety of criteria.

In this paper, we propose a implementation on Programming Protocol Independant Processors (*P4*) for Probing-Based Path Verification (PBV), able to do verify the route used for a packet. This is achieved by using a composition of hash functions on stateless core switches. It can then be checked by a trusted party that knows the `node_ids` independently. This work is intended to implement part of **PathSec** and is available online on (github.com/Henriquelay/polka-halfsiphhash).

2. Problem definition

A **PathSec** network consists of a logically centralized SDN Controller that selects routing paths and configures the nodes, edge nodes that embed metadata into packets, core nodes that execute basic packet forwarding with light signatures operations (proof-of-transit), and smart contracts running on a blockchain to support signatures verification and to

register path compliance. A unique Residue Number System-based is used to generate a `route_id`, from which a core node calculates the next hop, using the node's secret `node_id` [Martinello et al. 2024]. For auditability, the resulting multi-signed would then be sent to a smart contract and blockchain system which are out of scope for this paper.

2.1. Problem formalization

Let i be the source node (ingress node) and e be the destination node (egress node). Let the path from i to e $P_{i \rightarrow e}$ be a sequence of nodes: the path is defined by $P_{i \rightarrow e} = (i, s_1, s_2, \dots, s_{n-1}, s_n, e)$, s_n being the n -th core node in the path.

In SR protocols, the route up to the protocol boundary (usually, the SDN border) is defined in i . That is, i sets the packet header with enough information for each core node to calculate the next hop. Calculating each hop is done exploiting the Chinese Remainder Theorem [Dominicini et al. 2020], and is out of the scope of this paper.

The main problem we are trying to solve is path verification, that is, to have a way to ensure if the packet follows the path defined. A solution must be able to identify if the packet: (i) has passed through all nodes in the Path; (ii) has passed through the correct order of nodes; (iii) has **not** passed through any node that is **not** in the path.

More formally, given a sequence of nodes $P_{i \rightarrow e}$, and a captured sequence of nodes actually traversed P_j , a solution must identify if $P_{i \rightarrow e} = P_j$. Notably, it does not require validation, that is, it needs only to respond if $P_{i \rightarrow e} = P_j$ and does not need to know any of their contents or check their validity as a route.

2.2. Multi-signature model for Path Verification

To balance trace effectiveness and scalability, **PathSec** proposes using packets themselves as probes [Martinello et al. 2024] [Basat 2020], with a lightweight multi-signature, keeping the header size fixed. That is, each core node hashes the previous' node signature with a key only itself (and the Controller) can possess, and pass it forward, and all following core nodes will do the same until the edge node is reached.

Each node's execution plan is stateless. So, in terms of signature composition, a node n_i can be viewed as a function $f_{n_i}(x)$. A node can alter the header of the packet, which we will use to push information downstream and ultimately detect if the path taken is correct. **Está preciso e claro dizer que "which we will use to push information downstream"?**

In order to represent all nodes by the same function, we assign a distinct key value k for each node n_i , and use a bivariate function $f(k_{n_i}, x) = f_{n_i}(x)$. By using functions in two variables and enforcing one of the variables to have any value that's unique between nodes, we ensure that the function's result is unique for each switch as long as the function is collision resistant enough, that is, $f_{n_a}(x) \neq f_{n_b}(x) \iff y \neq z$.

Using function composition appropriate to use, as it preserves the order-sensitive property of the path, since $f \circ g \neq g \circ f$ in a general case. In our model, each node will execute a single function of this composition, using the previous node's output as input. In this way, $(f_{n_1} \circ f_{n_2} \circ f_{n_3})(x) = f(k_{n_3}, f(k_{n_2}, f(k_{n_1}, x)))$, f being a collision-resistant function, preferably a hash function.

3. Implementing and Testing a Probing-Based Path Verification (PBV)

PBV consists of sending special packets (probes) with path-verification capabilities. The path verification system used is describe in subsection 2.2. Some assumptions are made: (i) Each node is assumed to be secure, that is, no node will alter the packet in any way that is not expected. This is a common assumption in SDN networks, where a trusted party is the only entity that can alter the network state; (ii) Every link is assumed to be perfect, that is, no packet loss, no packet corruption, and no packet duplication; (iii) Protocol boundary is IPv4. This means that *PolKA* is only used inside this network, and only IPv4 is used outside; (iv) All paths are assumed to be valid and all information correct unless stated otherwise; (v) Hash collisions will not happen. Furthermore, this implementation is a proof-of-concept^{está certo chamar de proof-of-concept?}, and there is little concerns about performance and protocol implementation: only IPv4 is supported. The following subsections detail the setup and implementation.

3.1. Setup

P4 is a language used to program the data plane of network devices[Bosshart et al. 2014]. Behavioral Model 2 (*BMv2*) is a software switch that supports the *P4* language [Consortium]. Mininet is a network emulation tool that allows the creation of virtual networks with a controlled environment[Lantz et al. 2010]. Mininet-wifi¹ is a Mininet fork used in this work due to its compatibility with *BMv2*. Despite the name, no Wi-Fi emulating feature was used.

Scapy is a Python library that allows the creation and manipulation of packets[secdev 2003]. It is used to parse packets in the experiments and automate tests. The setup is shown in Figure 1. Controller is responsible for the network state and reference signature calculation, and the Tester triggering Mininet hosts to send packets, setup Scapy for parsing them and then asserting their signatures to Controller-calculated references.

3.2. Implementation

We define the header fields `timestamp` and `lhash` to indicate the uniquely-generated number and the current digest, respectively. Due to header incompatibility, probe packets operate using a different version number in the `version` field, so it is interoperable with *PolKA* by switching between probes and regular packets using the `version` header. *PolKA* packets use version `0x01`, and probe packets use version `0xF1`. Figure 2 shows the used topology used in the experiments where s_i is a switch, e_i is an edge node, and h_i is a host. The topology is a linear chain with 10 switches and 11 hosts. The links are bidirectional, but only one direction is shown for simplicity.

On edge nodes, if an IPv4 protocol `ethertype` field is detected (`0x0800`), it must be a packet from outside the network, so the edge is doing an ingress and *PolKA* headers need to be added. Let this process be called *encapsulation*; If a *PolKA* protocol `ethertype` field is detected (`0x1234`), it must be a packet from the network core, so the edge is doing an egress and the original IPv4 packet must be unwrapped. Let this process be called *decapsulation*.

¹github.com/intrig-unicamp/mininet-wifi

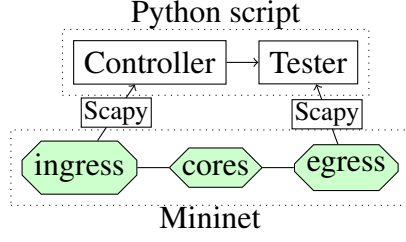


Figure 1: Experimentation setup

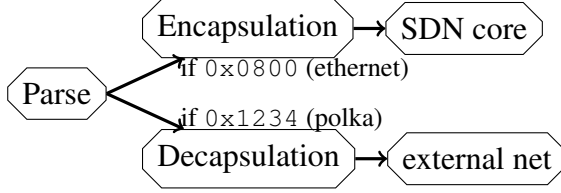


Figure 3: Edge ingress/egress steps

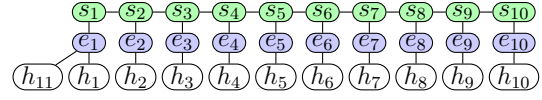


Figure 2: Baseline topology used in experimentation

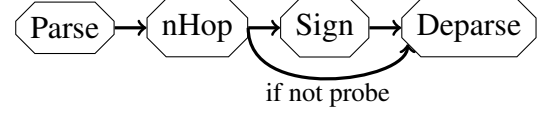


Figure 4: Data plane steps **Muito simples?**

PolKA headers consists of the route polynomial (`route_id`), along with `version`, `ttl` and `proto` (stores the original ethertype). `route_id` and hop calculation is out of the scope of this paper. During encapsulation, the packet can be made into a probe packet by setting the `version` header field to `0xF1` and further defining the 32-bit fields `timestamp` to any unique value and `l_hash` to `timestamp`.

On the core network, the hash function used is *SipHash-2-4-32* (*HalfSipHash*), which after calculating the next hop (`nhop`) and if the packet is a probe packet `l_hash` is computed as

$$l_hash \leftarrow \text{SipHash2-4-32}(\text{node_id} || \text{nhop} || \text{timestamp} || 0000000, l_hash)$$

Meaning that a 64-bit string made of concatenating `node_id`, `nhop`, `timestamp` with the remaining bits set to 0 for padding is used as key for the *HalfSipHash* function hashing `l_hash`.

4. Adversity scenarios

This section presents results for various tests representing different adversity scenarios. Each scenario is designed to evaluate the robustness and security of the system under specific attack or misconfiguration conditions. The scenarios include addition, detour, skipping, and out-of-order packet delivery. We define any switch that is not in the position defined in Figure 2 as an *attacker*, not only an intruder device, so it includes misconfigured switches, controller malfunction, and any other kind of network-affecting anomaly that results in these specified scenarios.

Table 1 shows the intermediary checksums for each scenario, highlighting the differences caused by the adversarial conditions, although in deployment, only the last `l_hash` is every used for verification. Note that a scenario may result in a different amount of hashes to compare due to different amount of intermediary hops, so the table isn't completely filled to represent a hop that does not exist on a particular route. For all scenarios, a ping from h_1 to h_{10} was performed, resulting in `route_id` = `0x707b3a1d61d1d0d8b9fc91e442d0360dfc8bba4`.

Reference		Addition		Skipping		Detour		Out-of-order	
s_0	0x61e8d6e7	s_0	0x61e8d6e7	s_0	0x61e8d6e7	s_0	0x61e8d6e7	s_0	0x61e8d6e7
s_1	0xd25dc935	s_1	0xd25dc935	s_1	0xd25dc935	s_1	0xd25dc935	s_1	0xd25dc935
s_2	0x245b7ac5	s_2	0x245b7ac5	s_2	0x245b7ac5	s_2	0x245b7ac5	s_2	0x245b7ac5
s_3	0xa3b38b83	s_3	0xa3b38b83	s_3	0xa3b38b83	s_3	0xa3b38b83	s_3	0xa3b38b83
s_4	0x26aee736	s_4	0x26aee736	s_4	0x26aee736	s_4	0x26aee736	s_4	0x26aee736
s_5	0xf9b47914	s_5	0xf9b47914			s_5	0xf9b47914		
		s_{add}	0x18c6d8d1					s_6	0x4b5a6c5a
s_6	0x18c6d8d1	s_6	0xb69b99ec	s_6	0x4b5a6c5a	s_{det}	0x250822a2	s_5	0xde3862a0
s_7	0xb69b99ec	s_7	0xfe6117f8	s_7	0x002346d3	s_7	0x40298bb9	s_7	0x648556ec
s_8	0xfe6117f8	s_8	0xc8d9fbde	s_8	0x7ec711aa	s_8	0xe13dcc9b	s_8	0x144e1d1b
s_9	0xc8d9fbde	s_9	0xa6293a25	s_9	0x5ee32b7b	s_9	0x1bf62c19	s_9	0x9e818f34
s_{10}	0xa6293a25	s_{10}	0xf4bcd07	s_{10}	0xc973a219	s_{10}	0xdd3a6675	s_{10}	0xf694bc

Table 1: Intermediary 1 hash for different test scenarios with the same seed

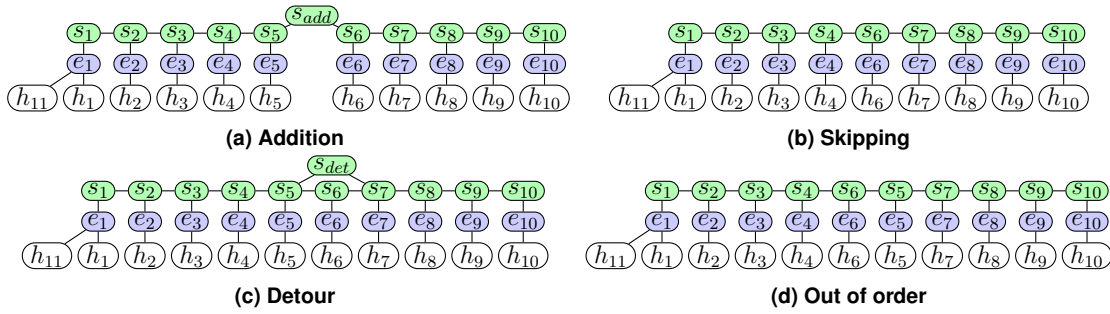


Figure 5: Topologies for tested scenarios

- Addition:** An attacker switch is added between s_5 and s_6 . It perfectly replicates the behavior of the original switch, but it does not have the same `node_id`, as it is a secret key, and the hashes will not match when validating the path past the attacker.
- Skipping:** A switch is removed between s_4 and s_6 . The packet will skip removed switch, and the hashes will not match when validating the path past the removed switch.
- Detour:** A switch is detoured between s_5 and s_7 into an attacker by hijacking the port that the packet should have taken exiting s_5 and delivering the packet in a new port to s_7 , posing as s_6 . The hashes will not match when validating the path past the attacker.
- Out-of-order:** Liks are change between s_5 and s_6 . The packet will be delivered to s_6 before s_5 , and the hashes will not match when validating the path past the attack.

4.1. Discussion

Algun nome melhor pra seção?

Recovering data from Mininet switches is not trivial, as it does not provide a straightforward way to extract data from the switches. This limitation makes it difficult to analyze the data in real-time, so our solution depends on sniffer libraries (Scapy) to capture packets and extract the metadata. This limitation could be overcome by having the switches natively output the metadata to our controller, which would allow for a more performant and scalable solution. It was not implemented this way to keep the implementation minimal.

The cryptographically secure hash solution is only secure when the key is a secret. The `node.id` secret is output in `ttl PolKA` header field for debugging and presentation purposes, and this implementation should never be deployed in a real-world scenario. If the key is compromised, the entire system is compromised, as a malicious actor can easily generate the same checksums and be undetected, essentially signing whichever packets they want. Having the switch entry port included in the verification would mitigate this issue, as the attacker would need to know the exact port the packet entered the switch.

5. Conclusion and Future Work

As the examples show, our solution is able to detect when a packet is not following the expected path for most cases, and can be used to detect misconfigurations in the network.

In next steps, more parts of PathSec[Martinello et al. 2024] system could be easily integrated, as all necessary information gets funnelled a Python script, making it simple to create proof-of-concept software.

To improve the system for a real-world scenario, the ingress edge needs to relay the ingressed package metadata directly to the controller, and the egress edge will report the final checksum directly to the blockchain. Having it not passing through the controller.

Replay attacks are not detectable by the current solution, as the switch entry port is not included in the verification. This allows an attacker to replay a packet to a different port, and the system will accept it as a valid packet. Including the entry port in the checksum would also be an appreciable increase in security, since it increases the number of targets an attacker would need to breach at the same time to be able to alter the path. It could be included in the *HalfSipHash* key, but a design decision is necessary: there are only 7 bits available, and the port number is 9 bits long. A possible solution would be to use the 8 least significant significant bits of the port number for both entry and exit port, limiting the number of ports checked before collision to 256, but this would be a reasonable trade-off for the increased security. So for a proof-of-concept, design was kept closed to the original **PathSec** description.

Further verification could be done to ensure the packet was unaltered during its path, as the current solution only verifies the path. This could be done by calculating the payload's hash in the verification process, but it could incur significant overhead and would not prove to be as usable, since the payload should be encrypted by an upper layer "upper layer" tá certo isso? most of the times that timing isn't critical (TLS). Manter isso? Não adiciona mto

An interesting work can be done to use a rotating key architecture to detect replay attacks. This is a difficult problem to solve, since the key must be rotated and shared in a way that does not break the network, and of course, a malicious actor cannot acquire it. The key must be shared between the nodes in a secure, atomic way to prevent the network to enter an irrecoverable state. Pareço estar me estendendo mto, mas ainda acho um problema interessante e queria anotar isso de alguma coisa. Mas é um problema buscando uma solução. Manter de alguma forma?

Using a keyless hash function, or even a keyed one with hardcoded keys on deployment could be studied, as it would remove the need for a seed timestamp, but it would require to be a cryptographic hash, otherwise the data used to hash (which needs

to include the `node_id`) could be derived from a switch's output. This would open ways to use a faster cryptographic hash function, such as xxHash's XXH3.

A timing analysis and stress tests can both be done to check if the incurred overhead is acceptable for the network, but this would require it to be implemented in hardware, not only in Mininet. The overhead needs to be low for it to be a viable solution, otherwise the network will drop packets. Some overhead is acceptable since a configurable ratio/schedule of *PolKA* packets are setup to be probe packets.

References

- [Basat 2020] Basat, B. (2020). Pint: Probabilistic in-band network telemetry. SIGCOMM '20, page 662–680, New York, NY, USA. Association for Computing Machinery.
- [Bosshart et al. 2014] Bosshart, P., Daly, D., Gibb, G., Izzard, M., McKeown, N., Rexford, J., Schlesinger, C., Talayco, D., Vahdat, A., Varghese, G., and Walker, D. (2014). P4: programming protocol-independent packet processors. *SIGCOMM Comput. Commun. Rev.*, 44(3):87–95.
- [Consortium] Consortium, P. L. Behavioral model. Accessed: 2024-09-26.
- [Dominicini et al. 2020] Dominicini, C., Mafioletti, D., Locateli, A. C., Villaca, R., Martinello, M., Ribeiro, M., and Gorodnik, A. (2020). Polka: Polynomial key-based architecture for source routing in network fabrics. pages 326–334.
- [Lantz et al. 2010] Lantz, B., Heller, B., and McKeown, N. (2010). A network in a laptop: rapid prototyping for software-defined networks.
- [Martinello et al. 2024] Martinello, M., Gomes, R. L., Borges, E. S., Layber, H. C., Bonella, V. B., Dominicini, C. K., Guimarães, R., Ribeiro, M., and Barcellos, M. (2024). Path-sec: Path-aware secure routing with native path verification and auditability. *Article*.
- [secdev 2003] secdev, B. P. (2003). Scapy. Accessed: 2024-09-26.